

## 提高级 CSP-S 第 8 套初赛模拟试题

一、单项选择题(共 15 题,每题 2 分,共计 30 分;每题有且仅有一个正确选项)

- 以下属于系统软件的是( )。  
A. c++编译器      B. 腾讯 QQ      C. CAD      D. 游戏软件
- 如果用一个字节来表示整数,最高位用作符号位,其他位表示数值。例如:00000001 表示+1,10000001 表示-1,试问这样表示法的整数 A 的范围应该是( )。  
A.  $-127 \leq A \leq 127$       B.  $-128 \leq A \leq 128$   
C.  $-128 \leq A < 128$       D.  $-128 < A < 128$
- 一篇文章有 200 个中文字符,50 个英文字符,20 个半角空格,如果只考虑存储这些文字本身需要( )个字节。  
A. 270 字节      B. 320 字节      C. 470 字节      D. 400 字节
- 下列属于网络模型的名称是( )。  
A. LAN      B. TCP/IP      C. FTP      D. SMTP
- 深度优先搜索时,如果不使用递归程序,一般需要用到的数据结构是( )。  
A. 链表      B. 队列      C. 栈      D. 散列表
- 学号为 1 到 30 的小朋友顺时针排成一圈,从 1 号小朋友开始顺时针报数,从数字 1 开始数下去,1,2,3,...,28,29,30,31,32,...,一圈又一圈,当数到数字 n 时,所在的小朋友学号是( )。  
A.  $(n-1) \% 30$       B.  $1+(n-1) \% 30$       C.  $(n+1) \% 30 - 1$       D.  $(n+1) \% 30$
- 一棵完全二叉树的结点总数为 41,其叶结点数为( )。  
A. 18 个      B. 19 个      C. 20 个      D. 21 个
- 给出 3 种排序:插入排序、冒泡排序、选择排序。这 3 种排序的时间代价分别是( )。  
A.  $O(n)$ 、 $O(n^2)$ 、 $O(\log(n))$       B.  $O(\log(n))$ 、 $O(n)$ 、 $O(n^2)$   
C.  $O(n^2)$ 、 $O(n)$ 、 $O(n)$       D.  $O(n^2)$ 、 $O(n^2)$ 、 $O(n^2)$
- 对以下关键字序列用快速排序法进行从小到大排序,速度最慢的情况是( )。  
A. {19,23,3,15,7,21,8}      B. {23,21,28,15,19,3,7}  
C. {19,7,15,28,23,21,3}      D. {3,7,15,19,21,23,28}
- 算法是指( )。  
A. 为解决问题而编写的计算机程序      B. 为解决问题而采取的方法与步骤  
C. 为解决问题而需要采用的计算机语言      D. 为解决问题而采用的计算方法
- 以下关于图的不正确说法是( )。  
A. 所有顶点的度数之和等于边数的 2 倍  
B. 所有顶点的度数之和不一定等于边数的 2 倍  
C. 任意一个图一定有偶数个奇点  
D. 在有向图中顶点的入度之和等于出度之和
- 在解决计算机主机与打印机之间速度不匹配时通常设置一个打印数据缓冲区,主要将要输出打印的数据依次写入该缓冲区,而打印机从该缓冲区中取出数据打印。该缓冲区应该是一个( )结构。  
A. 堆栈      B. 数组      C. 队列      D. 链表
- 6 个人分乘两辆不同的汽车,每辆车最多坐 4 人,则不同的乘车方法数为( )。  
A. 40      B. 50      C. 60      D. 70
- 为了实现两数交换,代码如下:  

```
void swapAB(int &a,int &b)
{
```

```

_____;
b=a-b;
a=a-b;
}

```

则空格内要填入的语句是( )。

- A.  $a=a+b$       B.  $a=a*b$ ;      C.  $a=a-b$ ;      D.  $a=a\&b$ ;

15. 某数列有 1000 个各不相同的数,由低到高按序排列,现要对该数列进行二分法检索,在最坏的情况下,需要检索( )个数据。

- A. 1000      B. 10      C. 100      D. 500

二、阅读程序(程序输入不超过数组或字符串定义的范围;判断题正确填√,错误填×;除特殊说明外,判断题 1.5 分,选择题 3 分,共计 40 分)

1.

```

01 #include<bits/stdc++.h>
02 using namespace std;
03 int n;
04 int a[1005],f[1005];
05 int main(){
06     int i,j;
07     cin>>n;int ans=0;
08     for (i=0;i<n;i++) cin>>a[i];
09     for (i=0;i<n;++i) {
10         f[i]=1;
11         for (j=i-1;j>=0;--j)
12             if (a[i]>a[j]) f[i]=max(f[i],f[j]+1);
13         ans=max(ans,f[i]);
14     }
15     cout<<ans<<"\n";
16     return 0;
17 }

```

输入保证是正整数  $n$  以及一个长度为  $n$  的数组,且相邻元素之间没有空格。如下例:

3

1 2 3

#### ●判断题

- (1) 若将第 06 行的“ $\text{int } i,j$ ;”改为“ $\text{int } i$ ;”,则程序会出现编译错误。( )  
 (2) 若将第 12 行的  $a[i]>a[j]$  改为  $a[i]<a[j]$ ,运行结果不变。( )  
 (3)  $f[i]$  表示以  $i$  结尾的最长不下降子序列长度。( )  
 (4) 若把第 09 行的“ $i=0$ ;”换成“ $i=1$ ;”,则程序运行结果不变。( )

#### ●选择题

- (5) 若  $n=100$ ,则  $\text{ans}$  的最大值是( )。  
 A. 90      B. 50      C. 99      D. 100  
 (6) 若  $n=15$ , $a=\{5,7,6,8,1,3,5,4,2,9,14,11,12,8,7\}$ ,则  $\text{ans}$  的值为( )。  
 A. 3      B. 7      C. 6      D. 5

2.

```

01 #include<iostream>
02 using namespace std;
03 const int maxn=100005;

```

```

04 int n;
05 int a[maxn];
06 int b[maxn];
07 void solve(int l,int r)
08 {
09     if (l==r) return;
10     int mid=(l+r)/2;
11     solve(l,mid);solve(mid+1,r);
12     int i=l,j=mid+1,k=1;
13     while (i<=mid && j<=r)
14     {
15         if (a[i]<=a[j]) b[k++]=a[i++];
16         else b[k++]=a[j++];
17     }
18     while (i<=mid) b[k++]=a[i++];
19     while (j<=r) b[k++]=a[j++];
20     for (int i=1;i<=r;i++) a[i]=b[i];
21 }
22 int main() {
23     cin>>n;
24     for (int i=1;i<=n;i++) cin>>a[i];
25     solve(1,n);
26     int ans=0;
27     for (int i=1;i<=n;i++) ans=ans+a[i]*i;
28     cout<<ans<<endl;
29     return 0;
30 }

```

● 判断题

(1) 即使 a 中有重复的数字,该程序仍能正常运行并输出正确答案。( )

(2) 当运行完 25 行后,a 数组单调不增。( )

● 选择题

(3) 当输入如下时,

n  
1 2 3 ..... n

输出答案为( )。

A.  $n * n$

B.  $n * (n+1)/2$

C.  $n * (n+1) * (n+2)/6$

D.  $n * (n+1) * (2 * n+1)/6$

(4) 该程序最坏情况下的时间复杂度为( )。

A.  $O(n)$

B.  $O(n^2)$

C.  $O(n \log n)$

D.  $O(n \log^2 n)$

(5) 当输入如下时,

5  
50 60 30 40 50

输出为( )。

A. 660

B. 760

C. 770

D. 720

(6) 下列各组输入获得的答案最大的是( )。

A. 5 30 50 60 70 70

B. 5 20 30 50 50 80

C. 5 10 10 70 70 50

D. 5 60 70 70 5 5

3.

```

01 #include<stdio.h>
02 char c[200][200];
03 int s[200],m,n;
04 void numara(){
05     int i,j,cod,nr;
06     for (j=0;j<n;j++){
07         nr=0;cod=1;
08         for (i=0;i<m;i++){
09             if (c[i][j]=='1'){
10                 if (!cod){cod=1;s[nr]++;nr=0;}
11             }
12             else{
13                 if (cod){nr=1;cod=0;}
14                 else nr++;
15             }
16         }
17         if (!cod) s[nr]++;
18     }
19 }
20 int main(){
21     int i,j;
22     scanf("%d%d\n",&m,&n);
23     for (i=0;i<m;i++) gets(c[i]);
24     numara();
25     for (i=1;i<=m;i++)
26         if (s[i]!=0) printf("%d %d ",i,s[i]);
27     return 0;
28 }

```

输入保证是正整数  $m$  和  $n$  以及一个  $m \times n$  的 01 矩阵,且相邻元素之间没有空格。如下例:

```

2 4
0101
1000

```

● 判断题

- (1) 若将第 21 行的“int i,j;”改为“int i;”,则程序会出现编译错误。( )
- (2) 程序最少输出 0 个数,最多输出  $2 \times m$  个数。( )
- (3)  $s[i]$  表示矩阵中有  $s[i]$  个列有  $i$  个 0。( )
- (4) 若把第 10 行的“cod=1;”和第 13 行的“cod=0;”全部换成“cod^=1;”,则程序运行结果不变。( )

● 选择题

- (5) 若  $m=100, n=100$ , 则  $s[1]$  的最大值是( )。  
 A. 100                      B. 500                      C. 1000                      D. 5000
- (6) 若  $m=95, n=95$ , 则  $s[5]$  的最大值是( )。  
 A. 760                      B. 950                      C. 1520                      D. 1900

### 三、完善程序(单选题,每小题3分,共计30分)

#### 1. (YY 的树) 题目给出 $m$ 颗非空有根(根节点编号为 1)区分左右儿子的二叉树。

将某一棵给出二叉树的叶子节点换成任意一棵二叉树称为一次变换,即给出的二叉树变换到新二叉树。

同一棵二叉树进行多次变换是指在每次变换后得到的新二叉树上继续进行变换。

若任意一棵大小超过 1 的二叉树都可以由给出的某一棵给出的二叉树通过有限次变换得到,则输出“Almost Complete”,否则输出“No”。

题目第一行首先读入一个数  $m$ ,表示二叉树的棵数。

接下来进行  $m$  次这样的操作。

读入一个数  $n$ ,接下来  $n$  行每行 2 个数  $l_i$  和  $r_i$  分别表示这棵二叉树第  $i$  个点的左右儿子编号,若没有左儿子,则  $l_i=0$ ,若没有右儿子,则  $r_i=0$ 。

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 const int N=2e6+5;
04 int n,m,tot;
05 vector<int>t;
06 int lc[N],rc[N];
07 inline bool isleaf(int u) { ① ;}
08 inline bool solve(vector<int>t) {
09     if (t.empty()) return false;
10     for (auto o : t) if (isleaf(o)) return true;
11     vector<int>t1,t2,t3,t4;
12     for (auto o : t) {
13         if (!lc[o]) t2.push_back(rc[o]);
14         if (!rc[o]) t1.push_back(lc[o]);
15         if ( ② ) {
16             if (isleaf(lc[o])) t3.push_back(rc[o]);
17             if (isleaf(rc[o])) t4.push_back(lc[o]);
18         }
19     }
20     ③ ;
21 }
22 int main() {
23     scanf("%d",&m);
24     for (int i=1;i<=m;++i) {
25         int n;scanf("%d",&n); ④ ;
26         for (int j=1;j<=n;++j) {
27             scanf("%d",&lc[tot+j]);if (lc[tot+j]) lc[tot+j]+=tot;
28             scanf("%d",&rc[tot+j]);if (rc[tot+j]) rc[tot+j]+=tot;
29         }
30         ⑤ ;
31     }
32     if (solve(t)) printf("Almost Complete\n");
33     else printf("No\n");
34     return 0;
35 }
```



- (1) ①处应该填( )。
- A. `lc[u]&&rc[u]`                      B. `!lc[u]&&!rc[u]`  
 C. `lc[u]||rc[u]`                        D. `!lc[u]||!rc[u]`
- (2) ②处应该填( )。
- A. `lc[o]&&rc[o]`                      B. `!lc[o]&&!rc[o]`  
 C. `lc[o]||rc[o]`                        D. `!lc[o]||!rc[o]`
- (3) ③处应该填( )。
- A. `return solve(t1)&&solve(t2)&&solve(t3)&&solve(t4)`  
 B. `return solve(t1)||solve(t2)||solve(t3)||solve(t4)`  
 C. `return (solve(t1)&&solve(t2))||(solve(t3)&&solve(t4))`  
 D. `return (solve(t1)||solve(t2))&&(solve(t3)||solve(t4))`
- (4) ④处应该填( )。
- A. `tot++`                                  B. `tot=0`  
 C. `t.push_back(1+tot)`                D. `t.push_back(1)`
- (5) ⑤处应该填( )。
- A. `tot+=n`                      B. `tot++`                      C. `tot=0`                      D. `t.clear()`
2. (逻辑游戏) 一个同学给了我一个逻辑游戏。他给了我图 1, 在这个图上, 每一段边界都已经进行了编号。我的任务是在图中画一条连续的曲线, 使得这条曲线穿过每一个边界一次且仅穿过一次, 而且曲线的起点和终点都在这整个区域的外面。这条曲线是允许自交的。

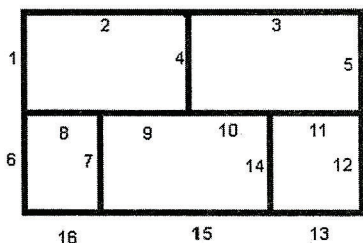


图 1

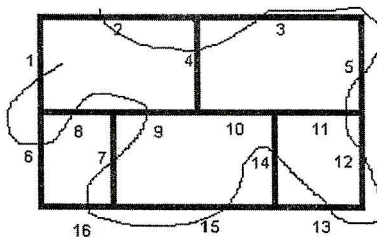


图 2

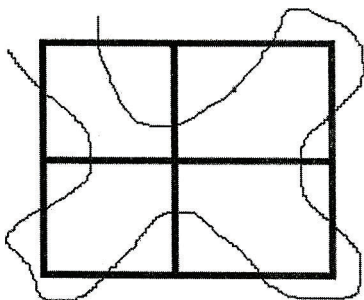


图 3

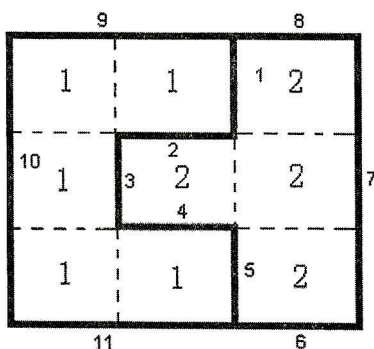


图 4

## 【输入】

输入的图形用一个  $n \times n$  的矩阵表示的。矩阵的每一个单元里有一个 0 到 255 之间 (包括 0 和 255) 的整数。处于同一个区域的单元里的数相同, 相邻区域的数不同 (但是不相邻的区域里的数可能相同)。

输入的第一行是  $n(0 < n < 100)$ 。以下的  $n$  行每行包括  $n$  个整数,分别给出对应的单元里的整数(这  $n$  个整数之间用空格分开)。图 4 给出了输入样例对应的图形。

【输出】

当可以画出满足题意的曲线的时候,输出“YES”;否则,输出“NO”。

输入样例:

```
3
1 1 2
1 2 2
1 1 2
```

输出样例:

YES

程序:

```
01 #include<stdio.h>
02 #include<math.h>
03 int orig,n,ns,a[102][102],bun;
04 int d[]={1,0,-1,0,0,1,①};
05 void plimba(int x,int y){
06     int i,xx,yy;
07     a[x][y]=-a[x][y];
08     if (abs(a[x-1][y])!=orig && (②!=a[x-1][y]
        || abs(a[x][y-1])!=orig)) ns++;
09     if (abs(a[x+1][y])!=orig && (a[x+1][y-1]!=a[x+1][y]
        || abs(a[x][y-1])!=orig)) ns++;
10     if (abs(a[x][y-1])!=orig && (③!=a[x][y-1]
        || abs(a[x-1][y])!=orig)) ns++;
11     if (abs(a[x][y+1])!=orig && (a[x-1][y+1]!=a[x][y+1]
        || abs(a[x-1][y])!=orig)) ns++;
12     for (i=0;i<4;i++){
13         xx=x+d[2*i];yy=y+④;
14         if (xx>=1 && xx<=n && yy>=1 && yy<=n && ⑤)
            plimba(xx,yy);
15     }
16 }
17 int main(){
18     int i,j;
19     bun=1;
20     scanf("%d",&n);
21     for (i=0;i<=n+1;i++)
22         for (j=0;j<=n+1;j++) a[i][j]=0;
23     a[0][0]=-1;a[n+1][0]=-1;
24     a[0][n+1]=-1;a[n+1][n+1]=-1;
25     for (i=1;i<=n;i++)
26         for (j=1;j<=n;j++) scanf("%d",&(a[i][j]));
27     for (i=1;i<=n;i++)
28         for (j=1;j<=n;j++){
29             if (a[i][j]>-1){
```

```
30          ns=0; ⑥ ;
31          plimba(i,j);
32          if (ns%2==1)bun=0;
33      }
34  }
35  if (bun) printf("YES\n");else printf("NO\n");
36  return 0;
37 }
```

(1) ①处应该填( )。

A. 0,-1      B. -1,0      C. 1,0      D. 0,1

(2) ②处应该填( )。

A. a[x][y-1]    B. a[x-1][y+1]    C. a[x-1][y-1]    D. a[x+1][y+1]

(3) ③处应该填( )。

A. a[x+1][y+1]    B. a[x-1][y]    C. a[x-1][y+1]    D. a[x-1][y-1]

(4) ④处应该填( )。

A. d[2\*i-1]    B. d[i]    C. d[i-1]    D. d[2\*i+1]

(5) ⑤处应该填( )。

A. abs(a[xx][yy]))!=orig    B. abs(a[xx][yy]))==orig

C. a[xx][yy]!=orig    D. a[xx][yy]==orig

(6) ⑥处应该填( )。

A. orig=a[i][j]    B. orig=-a[i][j]+bun

C. orig=bun    D. orig=abs(a[i][j])+bun